



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

Beezie Marketplace

July 2026

Prepared by
naman
Varun_Sharma

Introduction

A time-boxed security review of the **Beezie Marketplace** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About Beezie Marketplace

beezie_marketplace is a Solana Anchor program implementing a secondary marketplace for Metaplex Core NFTs within the Beezie collection. It supports two trade paths: fixed-price listings (seller lists an asset, buyer fills it via **buy_listing**) and escrowed bids (bidder deposits funds up front via **create_bid**, seller accepts via **accept_bid**). Listings authorize the marketplace's **escrow_authority** PDA as a Core transfer delegate on the asset rather than moving it into custody; bids escrow the payment token in a shared program-owned account until accepted or cancelled. Royalties are read from the asset's or collection's Metaplex Core Royalties plugin and split among creators at settlement. An admin/operator pair controls pausing, config updates, and moderation (force-cancelling listings); admin is bound to the program's upgrade authority at initialization.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

review commit hash - [fcb47619bc4c932a47490b6a4886a60976f60b45](#)

Scope

The program source lives in `solana-programs/programs/beezie_marketplace/`. The following files were in scope of the audit:

- `src/lib.rs` - entrypoint to the 11 instructions
- `src/royalties.rs` - MPL Core royalty enforcement and payout math
- `src/instructions/accept_bid.rs` - escrowed-bid settlement, value-transfer path
- `src/instructions/buy_listing.rs` - direct-buy settlement, value-transfer path
- `src/mpl_core_cpi.rs` - CPI into Metaplex Core for the NFT transfer
- `src/state/` - Config / Listing / Bid account layouts and PDA seeds
- `src/instructions/` - create/cancel/update listing and bid, initialize, transfer_admin, update_config

Value flows through `accept_bid`, `buy_listing`, and `royalties.rs`.

The following number of issues were found, categorized by their severity:

- Critical & High: 0 issues
- Medium: 1 issues
- Low: 6 issues
- Informational: 4 issues

Findings Summary

ID	Title	Severity
[M-01]	Collection authority can zero out seller proceeds by raising royalty basis points mid-trade	Medium
[L-01]	Seller can brick re-listing for the new owner by skipping listing cleanup on <code>accept_bid</code>	Low
[L-02]	Shared escrow account exposes all bidders to USDC's freeze authority	Low
[L-03]	Trading parties can evade royalties on trades priced below the basis point floor	Low
[L-04]	<code>update_listing</code> and <code>update_config</code> accept no-op updates and emit events reporting no real change	Low
[L-05]	<code>update_config</code> allows <code>operator</code> to be set to <code>Pubkey::default()</code> with no guard	Low
[L-06]	Malformed plugin registry data can waste compute on a single asset's parse	Low
[I-01]	Marketplace escrow authority can retain transfer rights over sold assets	Informational
[I-02]	Marketplace cannot support Token-2022 payment mints without a program upgrade	Informational

ID	Title	Severity
[I-03]	Redundant <code>payment_mint</code> validation in <code>process_create_bid</code>	Informational
[I-04]	<code>accept_bid</code> wastes compute re-deriving a PDA that is already stored on the listing account	Informational

Detailed Findings

[M-01] Collection authority can zero out seller proceeds by raising royalty basis points mid-trade

Severity

Impact: High

Likelihood: Low

Description

Neither `Listing` nor `Bid` stores any royalty information. `royalties::resolve_royalties()` is called fresh, at settlement time, inside both `process_buy_listing()` and `process_accept_bid()` — it reads whatever the Metaplex Core Royalties plugin says at that instant, not what it said when the listing or bid was created. `validate_royalties()` only enforces `basis_points <= 10_000`, so a royalty rate of 100% is a legal configuration value.

The only settlement-time consent guard either instruction has is on price: `buy_listing()` checks `expected_price == listing.price`; `accept_bid()` checks `expected_amount == bid.amount`. Neither checks the royalty split the seller implicitly accepted when listing at a given price.

```
// royalties.rs
let royalty_amount = calculate_royalty_amount(amount,
royalties.basis_points)?;
let seller_amount = amount.checked_sub(royalty_amount)?; // 0 if
basis_points == 10_000
```

If the Royalties plugin's governing authority raises `basis_points` to `10_000` between a listing or bid being created and it being filled, the seller receives zero proceeds on that trade — 100% of the sale price is redirected to the royalty-creator token account instead. The buyer pays the agreed price and receives the NFT as expected; only the seller is harmed, and has no way to detect or block the change before settlement executes.

Scenario

1. The Beezie collection's Royalties plugin is configured at `basis_points = 500` (5%), one creator at 100%.

- Alice lists Core NFT `C` via `create_listing(price = 1000)`, expecting roughly 950 units on sale.
- Before any buyer fills it, the collection's update authority raises the Royalties plugin to `basis_points = 10_000` via Metaplex Core directly — an action entirely outside this program.
- Bob calls `buy_listing(expected_price = 1000)`. The price check passes; `calculate_royalty_amount(1000, 10000)` returns `1000`, so `seller_amount = 0`.
- Bob pays 1000 units and receives the NFT. The creator token account receives all 1000 units. Alice receives 0.

Recommendations

It is recommended to snapshot a maximum acceptable royalty rate into the `Listing/Bid` account at creation time and enforce it at settlement, mirroring the existing `expected_price/expected_amount` guard.

```
pub struct Listing {  
    pub price: u64,  
+    pub max_royalty_bps: u16,  
    pub status: ListingStatus,
```

```
pub fn process_buy_listing<'info>(  
    ctx: Context<'info, BuyListing<'info>>,  
    args: BuyListingArgs,  
    ) -> Result<()> {  
    let royalties = royalties::resolve_royalties(...)?;  
+    let live_bps = royalties.as_ref().map_or(0, |r| r.basis_points);  
+    require!(live_bps <= ctx.accounts.listing.max_royalty_bps,  
    ErrorCode::RoyaltyExceedsCap);
```

Apply the same field and check to `Bid/create_bid/accept_bid`.

[L-01] Seller can brick re-listing for the new owner by skipping listing cleanup on `accept_bid`

Description

`Listing` PDAs are seeded solely by `[LISTING_SEED, asset]`, and nothing keeps a `Listing`'s `seller` field synchronized with the asset's actual current owner. `close_matching_listing()` inside `accept_bid()` only closes a matching listing if the seller supplies it as an optional extra account — nothing forces its inclusion. The same desync occurs if the asset changes hands through any path outside `buy_listing()/accept_bid()` entirely.

Once a stale `Active` listing exists for an asset the seller no longer owns, the new owner cannot call `create_listing()` for that asset — Anchor's `init` constraint rejects re-initializing the existing PDA —

and cannot self-serve a fix, since `cancel_listing()` requires the old seller's signature. Only an admin calling `cancel_listing_by_authority()` can free the PDA, blocking the new owner from listing that specific asset until then.

The documentation covers this scenario (`workflow.md`, `instructions.md`, and `opsec.md` all describe backend cleanup via `cancel_listing_by_authority()`), but the safety net is off-chain and not enforced by the program.

Recommendations

It is recommended to make listing cleanup mandatory rather than optional in `accept_bid()`, which will guarantee a matching listing is always closed when the underlying asset changes hands through this program.

```
- require!(
-   ctx.remaining_accounts.len() == royalty_account_count
-   || ctx.remaining_accounts.len() == royalty_account_count + 1,
-   ErrorCode::InvalidRoyaltyAccount
- );
+ let listing_exists = /* derive [LISTING_SEED, bid.asset] and check
+   account exists */;
+ let expected_len = royalty_account_count + if listing_exists { 1 } else {
+   0 };
+ require!(ctx.remaining_accounts.len() == expected_len,
+   ErrorCode::InvalidRoyaltyAccount);
```

[L-02] Shared escrow account exposes all bidders to USDC's freeze authority

Description

`escrow_token_account` is a single global ATA — the associated token account of `escrow_authority` for `payment_mint` — shared across every active bid in the marketplace. `create_bid()`, `cancel_bid()`, and `accept_bid()` all move funds through this one account. `payment_mint` is a classic SPL Token mint, and the team has confirmed USDC is the production payment token. USDC's freeze authority belongs to Circle, which has a documented history of freezing accounts for compliance and sanctions reasons. A freeze on the shared escrow account would therefore affect every bidder at once rather than a single participant.

This exposure exists for any protocol that accepts USDC and cannot be eliminated by this program — only its blast radius can be reduced. It is a property of the chosen payment token, not a defect in this program's logic, and is raised here for the team's awareness ahead of mainnet.

Recommendations

1. It is recommended to isolate escrow per bid (one PDA-owned token account per `Bid` instead of one shared account per marketplace), which will contain a freeze to the one affected bid instead of the

entire bid subsystem, at the cost of additional rent per bid.

```
#[account(  
    init,  
    payer = bidder,  
-    seeds = [ESCROW_AUTHORITY_SEED],  
+    seeds = [b"bid_escrow", bid.key().as_ref()],  
    bump,  
    token::mint = payment_mint,  
    token::authority = escrow_authority,  
)]  
pub bid_escrow_account: Account<'info, TokenAccount>,
```

2. It is recommended to close the per-bid escrow account and refund its rent back to the original payer (the bidder) whenever the bid resolves via `accept_bid()` or `cancel_bid()`, which will keep the per-bid isolation from point 1 rent-neutral for bidders instead of leaving them permanently out the extra rent each isolated account costs.

[L-03] Trading parties can evade royalties on trades priced below the basis point floor

Description

`calculate_royalty_amount()` floors `amount * basis_points / 10_000`. For any price `P` where `P * basis_points < 10_000`, the royalty rounds to exactly 0 and the seller keeps 100% of the on-chain proceeds. `price > 0` is the only floor guard on listing/bid creation.

For a collection with `basis_points = 250` (2.5%), any listing or bid priced at 39 base units or below pays zero royalty. A buyer and seller who agree to a real price off-chain and settle a token price under this threshold on-chain evade royalty payments entirely — a property inherent to any basis-points royalty system settled on-chain, since no such design can see or tax an off-chain side payment.

Recommendations

It is recommended to leave this as-is; no on-chain design can prevent off-chain side payments, and enforcing a minimum listing price would restrict legitimate low-value trades without closing the underlying evasion path.

[L-04] `update_listing` and `update_config` accept no-op updates and emit events reporting no real change

Description

Two state-mutating instructions apply caller-supplied values unconditionally, with no comparison against the value already stored. In both cases the write itself is harmless, but `updated_at` advances and an event fires that is indistinguishable from a genuine change — so any off-chain consumer keying off those signals is misled.

In `process_update_listing`, every `require!` in the validation chain passes when the seller submits the listing's existing `price` and `expires_at`. Nothing checks for an actual delta.

```
pub fn process_update_listing(ctx: Context<UpdateListing>, args:
UpdateListingArgs) -> Result<()> {
    require!(ctx.accounts.config.paused, ErrorCode::MarketplacePaused);
    require!(args.price > 0, ErrorCode::InvalidPrice);
    require!(
        ctx.accounts.listing.status == ListingStatus::Active,
        ErrorCode::ListingNotActive
    );
    let now = Clock::get()?.unix_timestamp;
    require!(
        args.expires_at == 0 || args.expires_at > now,
        ErrorCode::InvalidExpiration
    );
    ctx.accounts
        .listing
        .update_price(args.price, args.expires_at, now);
    emit_cpi!(ListingUpdated { ... });
    Ok(())
}
```

```
pub fn update_price(&mut self, price: u64, expires_at: i64, now: i64) {
    self.price = price;           // same value written back
    self.expires_at = expires_at; // same value written back
    self.updated_at = now;        // this DOES change, unconditionally
}
```

A seller can therefore repeatedly "bump" a listing's apparent freshness with a fully valid, zero-substance transaction, at the cost of ordinary transaction fees. Any off-chain system that treats `updated_at` or the `ListingUpdated` event as a signal of meaningful change — a "recently updated" sort order, or a price-change notification to watchers and bidders — can be triggered without the listing terms ever changing.

`MarketplaceConfig::update()` has the same gap on the admin path, applying `paused/operator` whenever `Some(_)` is supplied:

```
pub fn update(&mut self, operator: Option<Pubkey>, paused: Option<bool>,
now: i64) {
    if let Some(paused) = paused {
        self.paused = paused; // no check against the existing value
    }
}
```



```
    self.updated_at = now;
}
```

Calling `update_config(paused: Some(true))` while already paused succeeds, bumps `updated_at`, and emits a `MarketplaceConfigUpdated` event indistinguishable from a genuine transition. Indexers and monitoring that key off this event to detect real pause/unpause transitions will over-count or misreport them. No funds or protocol invariants are affected in either case.

Recommendations

It is recommended to require that the new value differ from the current one before applying it, in both instructions, which will make the event log an accurate record of real transitions only.

In `process_update_listing`:

```
require!(
  args.price != ctx.accounts.listing.price
  || args.expires_at != ctx.accounts.listing.expires_at,
  ErrorCode::NoChange
);
```

In `MarketplaceConfig::update`:

```
pub fn update(&mut self, operator: Option<Pubkey>, paused: Option<bool>,
now: i64) {
  if let Some(paused) = paused {
+    require!(paused != self.paused, ErrorCode::NoOpConfigUpdate);
    self.paused = paused;
  }
  self.updated_at = now;
}
```

[L-05] `update_config` allows `operator` to be set to `Pubkey::default()` with no guard

Description

`process_update_config` passes the caller-supplied `operator` argument straight through to `MarketplaceConfig::update` with no validation on its value:

```
pub fn process_update_config(ctx: Context<UpdateConfig>, args:
UpdateConfigArgs) -> Result<()> {
  let now = Clock::get()?.unix_timestamp;
  ctx.accounts.config.update(args.operator, args.paused, now);
}
```

```
...  
}
```

```
pub fn update(&mut self, operator: Option<Pubkey>, paused: Option<bool>,  
now: i64) {  
    if let Some(operator) = operator {  
        self.operator = operator;  
    }  
    ...  
}
```

If the admin calls `update_config` with `operator: Some(Pubkey::default())` — whether through a copy-paste error, an uninitialized-variable bug in off-chain tooling, or any other mistake that produces the all-zero pubkey — `config.operator` is silently overwritten with an address that has no corresponding private key and can never sign a transaction. Operator-gated maintenance, including stale-listing cleanup via `cancel_listing_by_authority()`, would then be unavailable until the admin sets a valid operator again.

Recommendations

Add an explicit guard in either `process_update_config` or `MarketplaceConfig::update` rejecting the zero address when `operator` is being set:

```
if let Some(operator) = args.operator {  
    require!(operator != Pubkey::default(), ErrorCode::InvalidOperator);  
}
```

[L-06] Malformed plugin registry data can waste compute on a single asset's parse

Description

`parse_royalties_at_plugin_registry()` reads `registry_len` as a raw `u32` from the Core asset/collection account's on-chain bytes and loops over it with no explicit sanity ceiling. Every read inside the loop is bounds-checked against the account's real byte length.

An inflated or corrupted `registry_len` value causes the loop to error out on the first out-of-bounds read rather than looping unboundedly, since the account's actual data length is the hard ceiling regardless of what the field claims. Worst case, buying or bidding on one specific asset with an unusually large plugin registry costs extra compute before reverting, scoped to that one asset only.

Recommendations

It is recommended to add an explicit upper bound check before the loop, which will fail faster and cheaper on malformed input without changing the program's actual security posture.

```
let registry_len = registry_cursor.read_u32()? as usize;  
+ require!(registry_len <= MAX_REASONABLE_PLUGINS,  
+ ErrorCode::InvalidRoyalties);  
for _ in 0..registry_len {
```

[I-01] Marketplace escrow authority can retain transfer rights over sold assets

Description

`create_listing()` approves `escrow_authority` as a Core `TransferDelegate` on the listed asset. `cancel_listing()` revokes this before closing the listing, but `buy_listing()`, `accept_bid()`'s listing cleanup, and `cancel_listing_by_authority()` do not — they close the listing without revoking the delegate.

This is not exploitable as written. `escrow_authority` only signs through this program, and the only place it can move an asset (`buy_listing()`) requires a fresh `Listing`, which only the asset's real current owner can create. The residual delegate approval is a cleanup gap rather than a live risk.

Recommendations

Call `revoke_transfer_delegate_v1()` in `buy_listing()`, `accept_bid()`'s listing cleanup, and `cancel_listing_by_authority()`, the same way `cancel_listing()` already does.

[I-02] Marketplace cannot support Token-2022 payment mints without a program upgrade

Description

`MarketplaceConfig.payment_mint` is typed `Account<'info, Mint>` from the legacy SPL Token program, and every transfer in the program uses `anchor_spl::token::transfer_checked()`. Anchor's owner check on this type requires the mint be owned by the legacy Token program, so a Token-2022 mint fails deserialization at `initialize()`.

This is a permanent design constraint, not an exploitable defect — it fails closed rather than open. It also means fee-on-transfer, transfer-hook, and interest-bearing token behaviors are structurally inapplicable to this program's `payment_mint`, since those extensions only exist on Token-2022. The constraint is stated in the project's documentation.

Recommendations

It is recommended to treat this as a deliberate product decision rather than a bug. Supporting a Token-2022 payment mint in the future would require a migration, not a configuration change.

[I-03] Redundant `payment_mint` validation in `process_create_bid`

Description

`payment_mint` is validated against `config.payment_mint` twice in the same instruction — once declaratively, once imperatively — with no code path between them that could cause the two checks to diverge.

The first check is in the `CreateBid` accounts struct, evaluated by Anchor during account validation, before the handler body ever runs:

```
#[account(
    constraint = payment_mint.key() == config.payment_mint @
    ErrorCode::InvalidPaymentMint,
)]
pub payment_mint: Box<Account<'info, Mint>>,
```

The second check is re-executed at the top of `process_create_bid`'s body, comparing the exact same two values:

```
require_keys_eq!(
    ctx.accounts.payment_mint.key(),
    ctx.accounts.config.payment_mint,
    ErrorCode::InvalidPaymentMint
);
```

Recommendations

Remove the redundant `require_keys_eq!` call from the body of `process_create_bid`.

[I-04] `accept_bid` wastes compute re-deriving a PDA that is already stored on the listing account

Description

`close_matching_listing()` calls `Pubkey::find_program_address()` to re-derive the listing PDA instead of reading the `Listing` account's own stored `bump` field and calling `create_program_address()` directly.

`find_program_address()` iterates bump seeds in the worst case, versus one direct call with a known-good bump, adding avoidable compute cost on a path that already runs per-creator royalty payout loops. The derived address is only ever used to validate the caller-supplied account before mutation, so this has no correctness or security consequence.

Recommendations

It is recommended to read the stored `bump` and call `create_program_address()` directly, which will remove the unnecessary seed search.

```
- let (expected_listing, _) = Pubkey::find_program_address(&[LISTING_SEED,
bid.asset.as_ref()], &crate::ID);
+ let expected_listing = Pubkey::create_program_address(&[LISTING_SEED,
bid.asset.as_ref()], &[listing.bump], &crate::ID)?;
```